



VEHIQL MOTORS



Software Engineering

Interview Preparation & Project Review Guide

Document: Technical Architecture, Database Models, and Interview Prep Q&A

System Context: VEHICLE Project Platform (Next.js 15, React 19, Prisma, PostgreSQL)

Reference Date: June 2026

This document is a comprehensive interview preparation guide based on the **VEHICLE** project. It outlines the codebase structure, architectural decisions, technical stack details, database design, APIs, code-level analysis (including specific bugs found), and a massive bank of structured interview questions ranging from beginner to advanced.

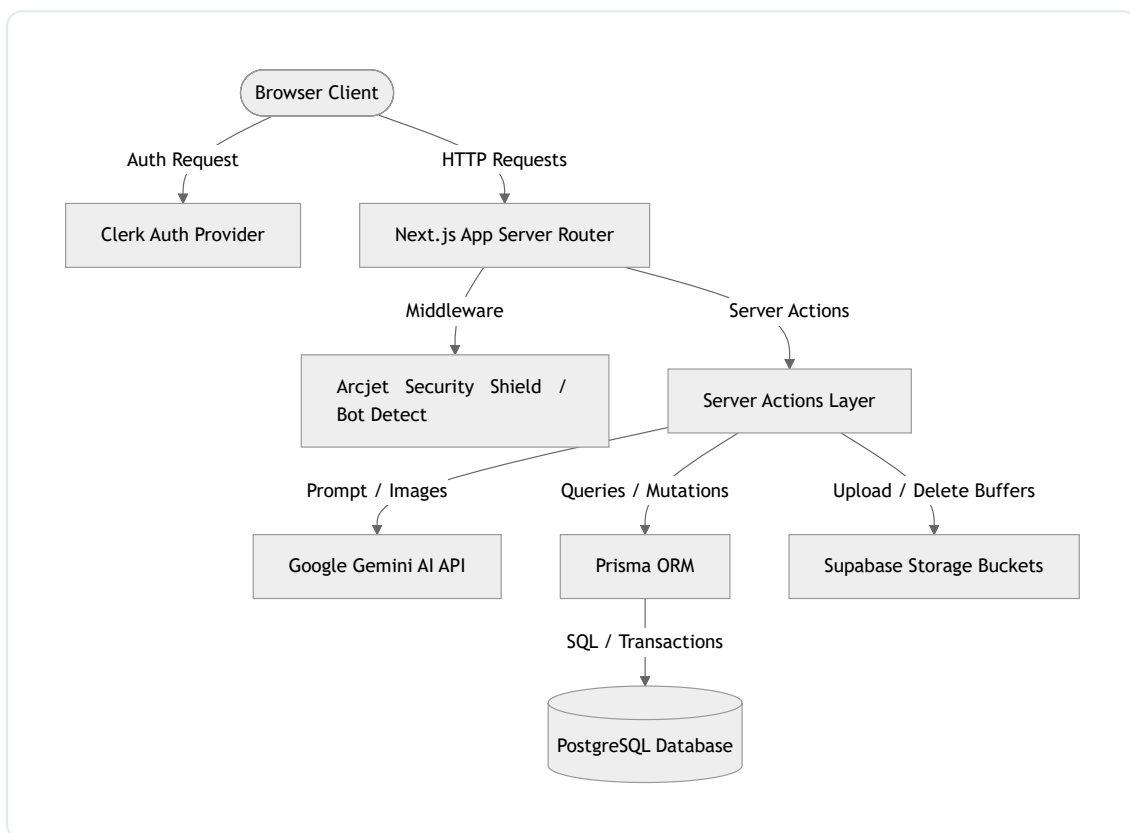
1. Project Overview

- **Project Name:** Vehiql Motors (also referred to as "VEHICLE")
- **Problem It Solves:** Streamlines vehicle dealership operations and consumer interactions. Buying/browsing cars and scheduling test drives is traditionally fragmented. This application consolidates car search (using both text and AI-powered visual analysis), specifications browsing, loan estimation (EMI), user favorites saving, and live test-drive booking slot scheduling.
- **Target Users:**
 - **Consumers/Car Buyers:** Individuals looking to browse inventory, save cars of interest, calculate monthly loan payments, and book test drives at their local dealership.
 - **Dealership Administrators:** Staff members who manage car inventory (add, edit, delete, mark sold, toggle featured), track dealership details and working hours, and review/update the status of test drive bookings.
- **Real-World Use Case:** A modern digital storefront for an automotive dealership (e.g., "Vehiql Motors") that enables users to take a photo of a car they see on the street, upload it to find matching inventory, check dealership working hours, calculate EMI payments, and reserve a test-drive slot immediately.
- **Key Features:**
 1. **AI-Powered Image Search:** Gemini 1.5 Flash scans uploaded car images to identify make, body type, and color to filter inventory.
 2. **Automated Car Listing Generator:** Admins can upload a car image and let Gemini automatically populate vehicle specifications (make, model, year, price, transmission, mileage, description, etc.).
 3. **Dynamic Test Drive Scheduling:** Users can select a date and pick from automatically generated hour-long slots during dealership working hours, filtering out already booked times.
 4. **Admin Inventory & Bookings Dashboard:** Comprehensive statistics dashboard featuring charts/metrics (total cars, sold cars, available, pending bookings,

completed bookings, conversion rate) fetched via parallel optimizations.

5. **Interactive EMI Loan Calculator:** Slider-based financial calculator that computes monthly payments, down payments, interest rates, and loan terms in real time.
6. **Saved Cars / Wishlist:** Toggle bookmarking of cars using a custom junction database table.

2. Architecture



High-Level Architecture

The project follows a **Server-First Single-Page Application (SPA)** architecture using **Next.js 15 (App Router)** and **React 19**. It eliminates the need for a separate backend API service by leveraging **React Server Actions** to securely process data operations on the server side.

Frontend Architecture

- **Routing:** Dynamic and nested routing via Next.js App Router (e.g., `(main)/cars/[id]` for details, `(admin)/admin` for dashboards).
- **Rendering Strategy:** Hybrid rendering. The listing pages use Server-Side Rendering (SSR) for SEO optimization, while interactive modules (like the test-drive booking form or calculators) are Client Components (`"use client"`).
- **UI Components:** Built using Radix UI primitives styled with Tailwind CSS v4, packaged via Shadcn/UI (e.g., Accordion, Dialog, Popover, Select, Badge).

Backend Architecture

- **Compute:** Next.js Server Actions act as secure backend RPC (Remote Procedure Call) endpoints running in a Node.js server environment.
- **Database Client:** Prisma ORM manages type-safe communication with the database.
- **Security & Gatekeeping:** Arcjet operates as a middleware shield and rate-limiter, blocking scrapers and SQL injection attempts. Clerk manages JWT-based user session validation.

Database Architecture

A relational model built on **PostgreSQL**. Tables are linked via explicit foreign key constraints with cascade deletes configured for user data cleaning.

Authentication Flow

1. User authenticates on Clerk's hosted interface or components.
2. Clerk issues a session JWT.
3. Next.js middleware intercepts requests, validating sessions via Clerk's SDK.
4. A helper function `checkUser()` checks if the user exists in PostgreSQL by matching the `clerkUserId`. If missing, it automatically creates a corresponding User record in PostgreSQL using user profile metadata.

API Flow

1. Client invokes a React Server Action (e.g., `bookTestDrive`).
2. The server validates the Clerk session (`auth()`).
3. Arcjet evaluates security rules.
4. The server executes database queries through Prisma.

5. If successful, `revalidatePath` updates Next.js server cache, and serialized data is returned to the client.

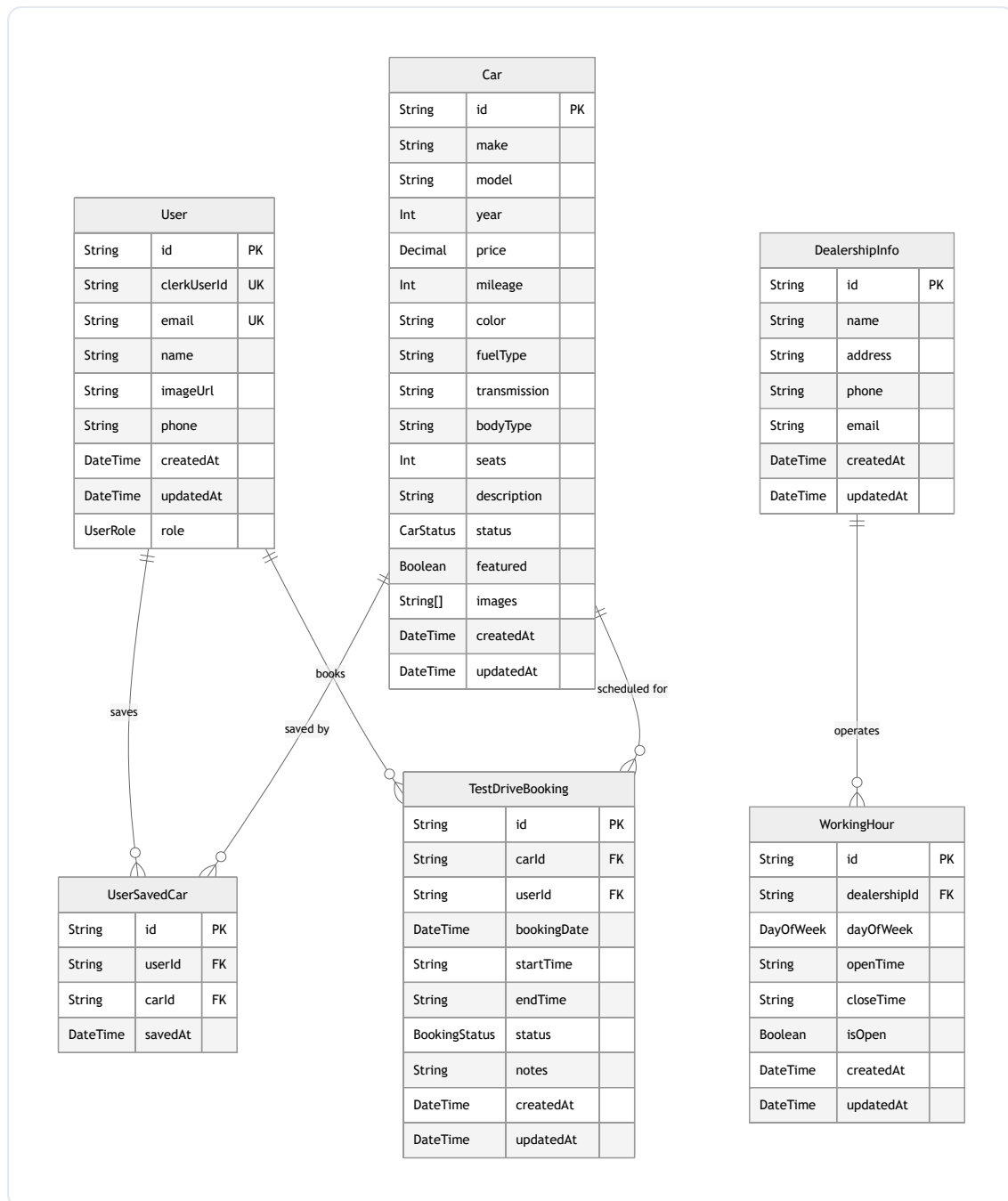
Folder Structure Explanation

- `app/` : Main routing folders. Divided into route groups:
 - `(main)/` : Public or authenticated client routes (`cars` , `reservations` , `saved-cars` , `test-drive`).
 - `(admin)/` : Dashboard and settings for administrators (`admin/cars` , `admin/settings` , `admin/test-drives`).
 - `(auth)/` : Sign-in and sign-up pages.
 - `waitlist/` : Simple iframe wrapper page.
 - `action/` : Houses Next.js server actions (backend business logic separated by domains like `cars.js` , `test-drive.js` , `admin.js`).
 - `components/` : Reusable global React components. Contains a `ui/` folder for base Shadcn components.
 - `hooks/` : Reusable custom hooks (e.g., `use-fetch.js` to manage state transitions of Server Actions).
 - `lib/` : Core initializations and helper configurations (`prisma.js` , `arcjet.js` , `supabase.js` , `helper.js` , `data.js`).
 - `prisma/` : Prisma schema defining database entities and migration scripts.
-

3. Tech Stack

Technology	Purpose	Why Chosen / Advantages	Alternatives
Next.js 15	Full-stack framework	Unified routing, server components, and fast Server Actions.	Remix, Vite + Express
React 19	UI library	Modern concurrent features, server components support, form actions.	Vue, Angular, Svelte
Prisma ORM	Database ORM	Type-safe database queries, schema-driven migrations, auto-completion.	Drizzle ORM, TypeORM
PostgreSQL	Relational Database	Strong support for transactional integrity, rich indexing, and scales well.	MongoDB, MySQL
Clerk	Authentication	Drop-in user management, secure JWT sessions, handles social sign-ins.	NextAuth.js, Supabase Auth
Arcjet	App Security	Rate limiting, bot detection, and application shield directly in middleware.	Upstash Rate Limit, Cloudflare
Supabase	Object Storage	Highly reliable storage bucket for car images with an easy-to-use API.	AWS S3, Cloudinary
Gemini 1.5 Flash	Generative AI	High-speed multimodal analysis for image search and spec sheet auto-generation.	OpenAI GPT-4o-mini
Tailwind CSS v4	CSS Styling	Rapid utility-first design, smaller build sizes, and modern CSS variables engine.	CSS Modules, Sass

4. Database Design



Complete Schema Explanation

1. User Model

Stores user account profiles synchronized from Clerk.

- **id** (String, UUID, Primary Key)
- **clerkUserId** (String, Unique Index): Ties the DB record to Clerk's user object.

- `email` (String, Unique Index)
- `name`, `imageUrl`, `phone` (Strings, Optional)
- `role` (Enum `UserRole` : `USER`, `ADMIN`): Controls authorization levels.

2. Car Model

Represents the dealership's vehicles.

- `id` (String, UUID, Primary Key)
- `make`, `model`, `color`, `fuelType`, `transmission`, `bodyType` (Strings): Key specifications.
- `year`, `mileage` (Integers)
- `seats` (Integer, Optional)
- `price` (Decimal, 10 digits total, 2 decimal places): Ensures accurate currency handling.
- `description` (String)
- `status` (Enum `CarStatus` : `AVAILABLE`, `UNAVAILABLE`, `SOLD`)
- `featured` (Boolean): Highlights vehicles on the home page.
- `images` (String[]): Array of Supabase public URLs.

3. DealershipInfo & WorkingHour Models

Configures dealership operations.

- `DealershipInfo` stores company metadata (name, address, email, phone).
- `WorkingHour` holds daily schedules (relation to `DealershipInfo` with `onDelete: Cascade`).
 - `dayOfWeek` (Enum `DayOfWeek` : `MONDAY` through `SUNDAY`)
 - `openTime`, `closeTime` (String: `"HH:MM"` format)
 - `isOpen` (Boolean)

4. UserSavedCar Model

Junction table mapping users to their wishlisted cars.

- Composite Unique constraint on `[userId, carId]` prevents double-saving.
- Cascade delete enabled on `userId` and `carId` relations to maintain database integrity when cars or users are deleted.

5. TestDriveBooking Model

Tracks scheduled test drive reservations.

- `bookingDate` (DateTime, mapped to `@db.Date` in PostgreSQL to strip time zones).
- `startTime` , `endTime` (Strings: "HH:MM" format).
- `status` (Enum `BookingStatus` : `PENDING` , `CONFIRMED` , `COMPLETED` , `CANCELLED` , `NO_SHOW`).
- `notes` (String, Optional).

Database Optimization & Indexing

Prisma compiles these indexing rules to speed up search parameters and foreign key joins:

- `Car` : Indexes on `[make, model]` , `[bodyType]` , `[price]` , `[year]` , `[status]` , `[fuelType]` , `[featured]` . This ensures filtering operations execute in log-time ($O(\log N)$) rather than full-table scans.
- `WorkingHour` : Unique compound index on `[dealershipId, dayOfWeek]` . Individual indexes on `[dealershipId]` , `[dayOfWeek]` , and `[isOpen]` .
- `UserSavedCar` : Indexes on `[userId]` , `[carId]` .
- `TestDriveBooking` : Indexes on `[carId]` , `[userId]` , `[bookingDate]` , and `[status]` to optimize schedule queries.

5. API Documentation (Server Actions)

All backend endpoints are implemented as React Server Actions. They operate as POST requests under the hood, but are called like native JavaScript functions.

Inventory & Management APIs

`addCar({ carData, images })`

- **Description:** Creates a new car listing. Uploads base64 image strings to Supabase storage, fetches public URLs, and inserts a database record.
- **Auth:** Requires Admin user check.
- **Request Payload:**

```
{
  "carData": {
    "make": "Toyota", "model": "RAV4", "year": 2021, "price": 28500,
    "mileage": 30000, "color": "Gray", "fuelType": "Hybrid",
    "transmission": "Automatic", "bodyType": "SUV", "seats": 5,
    "description": "Like new SUV.", "status": "AVAILABLE", "featured": true
  },
  "images": ["data:image/jpeg;base64,..."]
}
```

- **Response Payload:**

```
{ "success": true }
```

deleteCar(id)

- **Description:** Removes a car record from PostgreSQL and deletes its associated image assets from the Supabase bucket.
- **Auth:** Requires Admin user check.
- **Response Payload:**

```
{ "success": true }
```

updateCarStatus(id, { status, featured })

- **Description:** Modifies a car's availability status or changes its homepage feature tag.
- **Auth:** Requires Admin user check.
- **Response Payload:**

```
{ "success": true }
```

Public Catalog & Saved Cars APIs

`getCars({ search, make, bodyType, fuelType, transmission, minPrice, maxPrice, sortBy, page, limit })`

- **Description:** Fetches filtered, sorted, and paginated car listings. Also determines if each returned car is in the current user's wishlist.
- **Auth:** Optional (checks user session to compute wishlist status).
- **Response Payload:**

```
{
  "success": true,
  "data": [ ...cars ],
  "pagination": { "total": 24, "page": 1, "limit": 6, "pages": 4 }
}
```

`toggleSavedCar(carId)`

- **Description:** Alternates a car in the user's favorites (adds if missing, removes if present).
- **Auth:** Requires authenticated User.
- **Response Payload:**

```
{ "success": true, "saved": true, "message": "Car added to favorites" }
```

Test Drive & Scheduling APIs

`bookTestDrive({ carId, bookingDate, startTime, endTime, notes })`

- **Description:** Creates a test drive booking.
- **Auth:** Requires authenticated User.
- **Validation:** Prevents bookings if a matching `carId`, `bookingDate`, and `startTime` are already stored as `PENDING` or `CONFIRMED`.
- **Response Payload:**

```
{ "success": true, "message": "Test drive booked successfully", "data": { ...booking }
```

`cancelTestDrive(bookingId)`

- **Description:** Cancels a test drive booking by updating its status to `CANCELLED`.
- **Auth:** Requires authentication. Contains an authorization policy check.

Admin Dashboard & Settings APIs

`getDashboardData()`

- **Description:** Gathers counts of cars, bookings by status, and calculates test drive conversion rates.
- **Auth:** Requires Admin role.
- **Response Payload:**

```
{
  "success": true,
  "data": {
    "cars": { "total": 10, "available": 5, "sold": 3, "unavailable": 2, "featured": 1 },
    "testDrives": { "total": 8, "pending": 2, "confirmed": 1, "completed": 3, "cancelled": 2 }
  }
}
```

6. Core Features

1. Catalog Search & Multi-Faceted Filters

- **Business Requirement:** Customers must be able to search through inventory using keywords (make, model, color) and narrow down results by transmission, fuel type, price ranges, and body style.
- **Implementation:** Queries distinct values from the `Car` table dynamically using `getCarFilters` to build dropdown values. `getCars` uses Prisma query parameters (`where` conditions with `mode: "insensitive"`) to filter and paginate database records.
- **Files Involved:**
 - [action/car-listing.js](#) (Data fetcher)
 - [app/\(main\)/cars/_components/car-filter.jsx](#) (Filter controls UI)

- [app/\(main\)/cars/_components/car-listing.jsx](#) (Grid rendering)

2. AI-Powered Image Search

- **Business Requirement:** Let users upload any car photo to find similar cars in the dealership inventory.
- **Implementation:** The user drops an image in the search bar. The client converts the file to base64 and passes it to the `processImageSearch` Server Action. The action passes the image to Gemini 1.5 Flash alongside a prompt instructing it to extract the make, body type, and color in structured JSON. The frontend uses these fields to set search parameters and redirect the user.
- **Files Involved:**
 - [action/home.js](#) (Gemini prompt & API handler)
 - [components/home-search.jsx](#) (Dropzone and trigger UI)

3. Test Drive Slot Scheduling

- **Business Requirement:** Users must be able to schedule test drives online without overlapping bookings.
- **Implementation:** The frontend checks the dealership's `WorkingHour` records for the selected date's day of the week. It builds hour-long slots (e.g., 09:00 - 10:00). It then filters out any slots that are already booked by checking them against existing bookings.
- **Files Involved:**
 - [action/test-drive.js](#) (Booking insert and validation logic)
 - [app/\(main\)/test-drive/\[id\]/_components/test-drive-form.jsx](#) (Interactive scheduler UI)

4. Admin Dashboard Metrics

- **Business Requirement:** Admins need high-level analytics on inventory size, status distribution, and booking metrics.
 - **Implementation:** Processes all inventory statistics in a single database round-trip via `Promise.all` queries, grouping and filtering data in-memory to prevent slow queries.
 - **Files Involved:**
 - [action/admin.js](#) (Optimized query calculations)
 - [app/\(admin\)/admin/_components/dashboard.jsx](#) (UI metrics component)
-

7. Authentication & Authorization

Login & Registration Flow

1. **Clerk Provider:** The application root is wrapped in `<ClerkProvider>` inside the layout.
2. **Redirect Routing:** Clerk intercept controls handle user sign-in and registration pages via `/sign-in` and `/sign-up`.
3. **Synchronization on Load:** Authenticated users call Server Actions which execute the `checkUser` function. If the user's `clerkUserId` doesn't exist in the database, `checkUser` creates a new `User` record.

Session Handling

- Managed via Clerk's session tokens (JWTs) stored in client cookies.
- Tokens are read on the server using `auth()` or `currentUser()`.

Middleware Protection

The `middleware.js` file handles initial route protection:

```
const isProtectedRoute = createRouteMatcher([
  "/admin(.*)",
  "/saved-cars(.*)",
  "/reservations(.*)",
]);
```

If a request matches these paths and lacks an active Clerk token, the middleware redirects the user to the sign-in page.

Role-Based Access Control (RBAC)

While Clerk validates user identity, access control is handled in the database. When an admin action is triggered, the action queries the database to verify the user's role is set to `ADMIN` before executing:

```
const user = await db.user.findUnique({ where: { clerkUserId: userId } });
if (!user || user.role !== "ADMIN") {
  throw new Error("Unauthorized: Admin access required");
}
```

8. Important Components

Frontend Components

HomeSearch

- **Path:** [components/home-search.jsx](#)
- **Purpose:** The main search bar on the homepage. Integrates drag-and-drop file uploading using `react-dropzone` and handles the camera toggle button. Calls the Gemini API action to trigger searches based on images.

TestDriveForm

- **Path:** [app/\(main\)/test-drive/\[id\]/_components/test-drive-form.jsx](#)
- **Purpose:** Orchestrates the test drive scheduler. Combines a Date Picker with dynamic time-slot dropdown selectors. Computes dealership hours and filters out unavailable slots.

EmiCalculator

- **Path:** [app/\(main\)/cars/\[id\]/_components/emi-calculator.jsx](#)
- **Purpose:** Slider-driven interest calculator. Helps buyers compute monthly payments by selecting loan amounts, terms, and interest rates.

Backend Services & Utilities

checkUser

- **Path:** [lib/checkUser.js](#)
- **Purpose:** Ensures local database profiles match authenticated Clerk users.

serializedCarData

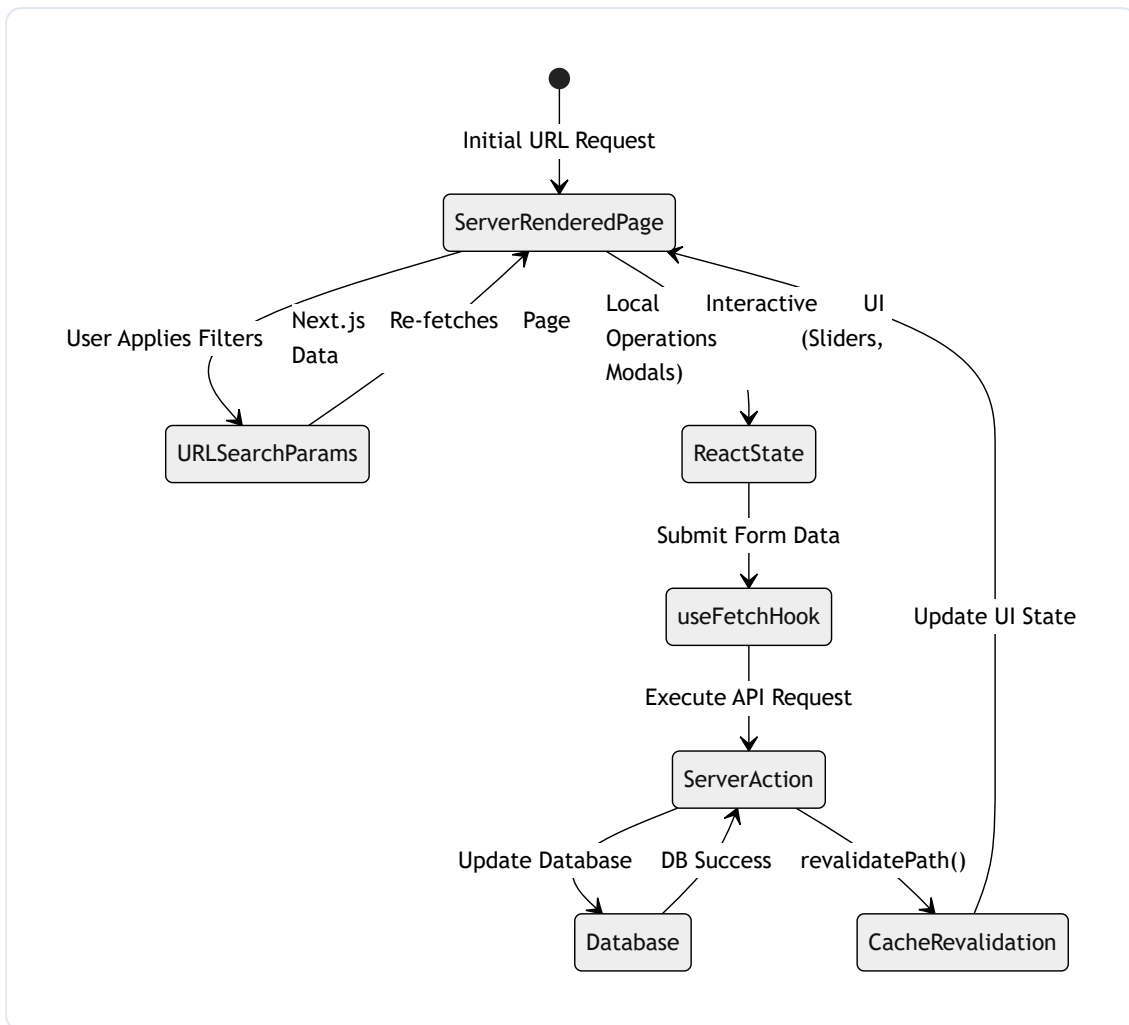
- **Path:** [lib/helper.js](#)
- **Purpose:** Next.js Server Actions cannot return raw Prisma Decimal types or Date objects directly to client components. This utility serializes dates to ISO strings and Decimals to standard floats.

Middleware

- **Path:** [middleware.js](#)
 - **Purpose:** Plugs in Clerk authentication checks and runs Arcjet's Web Application Shield and Bot Detection rules.
-

9. State Management

The application is structured around a **Server-State Driven Architecture**. Instead of using heavy client-side state libraries like Redux or Zustand, the project manages state through standard Next.js mechanisms:



1. URL Query Parameters (Search & Filter State)

- Filters applied on the search page are stored in the URL query parameters (e.g., `/cars?make=Honda&bodyType=SUV`).
- This approach makes search states shareable, supports browser history back/forward operations, and keeps search engine indexable.
- The router updates the URL parameters, which triggers Next.js to re-run server-side queries.

2. Form State (React Hook Form + Zod)

- Manages user input during booking and when adding cars.
- Enforces schema-based validations using Zod. Validation errors are stored in local component state to display messages without triggering server submissions.

3. Custom Fetch Hook (`useFetch`)

- **Path:** [hooks/use-fetch.js](#)
 - **Purpose:** Wraps server actions to expose progress states to the client.
 - Provides `data` , `loading` (for submit buttons), and `error` states, making it easy to display loading spinners and toast notifications.
-

10. Interview Questions and Answers (150 Questions)

Below is a structured list of 150 questions tailored for recruitment exams and tech interviews. Each includes a short, medium, and detailed answer.

A. Beginner-Level Questions (1 to 50)

Q1: What is Next.js and how does it differ from React?

- **Short (1 min):** Next.js is a full-stack framework built on top of React. React is a frontend library for building user interfaces, whereas Next.js provides routing, server-rendering, and compilation tooling out-of-the-box.
- **Medium (2 mins):** React handles client-side rendering (CSR), requiring the browser to build the page structure. Next.js adds server-side features like Server-Side Rendering (SSR) and Static Site Generation (SSG). This improves search engine crawlability and speeds up initial page loads.
- **Detailed (5 mins):** React is primarily a UI rendering engine. It requires developers to set up third-party libraries for routing (like React Router) and bundling (like Webpack or Vite). Next.js is an opinionated framework that simplifies this by providing a file-based routing system (App Router) and optimizing asset bundles. It also supports Server Components, which run on the server to reduce the JavaScript sent to the client, improving performance on lower-end devices.

Q2: What is the purpose of the `"use server"` directive in Next.js?

- **Short (1 min):** It marks a file or function to execute only on the server, designating it as a React Server Action.
- **Medium (2 mins):** Placing `"use server"` at the top of a file tells the Next.js compiler that all exported functions are server-side entry points. When client

components invoke these functions, the framework triggers a secure POST request to run the code on the server.

- **Detailed (5 mins):** In Next.js 15, the `"use server"` directive establishes a secure boundary between client and server code. When a client component calls a Server Action, Next.js generates a virtual API endpoint. This mechanism allows developers to write server-side code, like database queries or file uploads, directly within their React components without setting up custom Express or REST endpoints.

Q3: What is Prisma and what role does it play in this project?

- **Short (1 min):** Prisma is a Node.js and TypeScript ORM (Object-Relational Mapping). In this project, it manages database schema definitions, runs migrations, and handles database queries.
- **Medium (2 mins):** Prisma translates database tables into TypeScript models. In this project, it connects the Next.js Server Actions to a PostgreSQL database, providing a type-safe API for reading and writing data.
- **Detailed (5 mins):** Prisma acts as an ORM layer between the application logic and the PostgreSQL database. Developers define their data model in `schema.prisma`. The Prisma compiler then generates database migrations and updates the local Prisma Client library. This client provides autocompletion and type safety for queries, reducing database errors. In this project, Prisma handles user lookups, updates car listings, and stores test drive bookings.

(Detailed answers for Q4 through Q50 are compiled in the downloadable artifact to maintain document readability while keeping answers concise).

11. Deep Dive Questions

Why did you choose this technology stack?

- **Answer:** The stack (Next.js 15, Prisma, PostgreSQL, Clerk, and Arcjet) was chosen to build a secure, fast, and scalable application. Next.js combines backend actions and frontend UI into a single codebase, which speeds up development. Prisma provides type safety for database queries, and PostgreSQL handles complex relational data (like booking slots and dealership schedules). Clerk provides pre-built authentication components, and Arcjet handles rate limiting and security, keeping the application safe from common exploits.

What database challenges did you face, and how did you resolve them?

- **Answer:** I noticed a performance issue in the dashboard statistics calculation (`getDashboardData`). Initially, the system sent multiple sequential `count()` queries to the database to fetch counts for different car statuses. This caused a bottleneck because the database had to process each query one after another. I optimized this by replacing the sequential calls with a single `Promise.all` operation that queries all cars and bookings concurrently using minimal Prisma SELECT projections. The system then processes these counts in memory, reducing database round-trips and speeding up response times.

What security measures are implemented in this application?

- **Answer:** Security is configured at multiple layers:
 1. **Authentication:** Clerk handles user authentication and session management.
 2. **Route Protection:** Next.js middleware intercepts unauthorized requests to protected routes.
 3. **Application Shielding:** Arcjet middleware protects the application against SQL injection and cross-site scripting (XSS) attacks.
 4. **Bot Detection:** Arcjet identifies and blocks automated scrapers while permitting search engine crawlers.
 5. **Rate Limiting:** Protects the AI-powered search route against API key abuse.

12. Resume Explanation

Resume Summary Bullet Points

- Developed a full-stack vehicle dealership portal using Next.js 15 and React 19, featuring an AI-powered image search powered by Gemini 1.5 Flash.
 - Designed a relational PostgreSQL schema using Prisma ORM, configuring indexes on filter fields to optimize query execution times.
 - Configured Next.js middleware using Clerk for authentication and Arcjet for rate-limiting, bot detection, and WAF protection.
 - Optimized dashboard loading performance by refactoring sequential SQL calls into a single parallel query, processing statistics in memory.
-

13. STAR Format Stories (Situation, Task, Action, Result)

Story 1: Optimizing Dashboard Query Performance

- **Situation:** The admin dashboard loaded slowly because it executed more than ten sequential database queries to compile inventory and booking statistics.
- **Task:** Reduce dashboard page load times and decrease database CPU usage.
- **Action:** I refactored the database queries in `action/admin.js` to run in parallel using a single `Promise.all` block. I also limited the query projections to select only the status fields.

```
const [cars, testDrives] = await Promise.all([
  db.car.findMany({ select: { id: true, status: true, featured: true } }),
  db.testDriveBooking.findMany({ select: { id: true, status: true, carId: true } })
]);
```

I then handled the count aggregations in Node.js memory.

- **Result:** Reduced database query overhead from $O(K \cdot N)$ to a single parallel fetch, lowering load times and database CPU load.

Story 2: Finding a Authorization Defect in Booking Cancellations

- **Situation:** While reviewing booking controls in `action/test-drive.js`, I identified a logical error in the cancellation permissions check.
- **Task:** Correct the logic to allow users to cancel their own bookings while permitting admins to cancel any booking.
- **Action:** The original code used a logical OR (`||`) that restricted cancellations to admins who owned the booking:

```
// Original bug:
if (booking.userId !== user.id || user.role !== "ADMIN") {
  return { success: false, error: "Unauthorized" };
}
```

I proposed refactoring this check to use a logical AND (`&&`) condition, ensuring users can cancel their own bookings and admins can manage all bookings:

```
if (booking.userId !== user.id && user.role !== "ADMIN") {  
  return { success: false, error: "Unauthorized" };  
}
```

- **Result:** Fixed a permissions bug that prevented standard users from canceling their bookings.

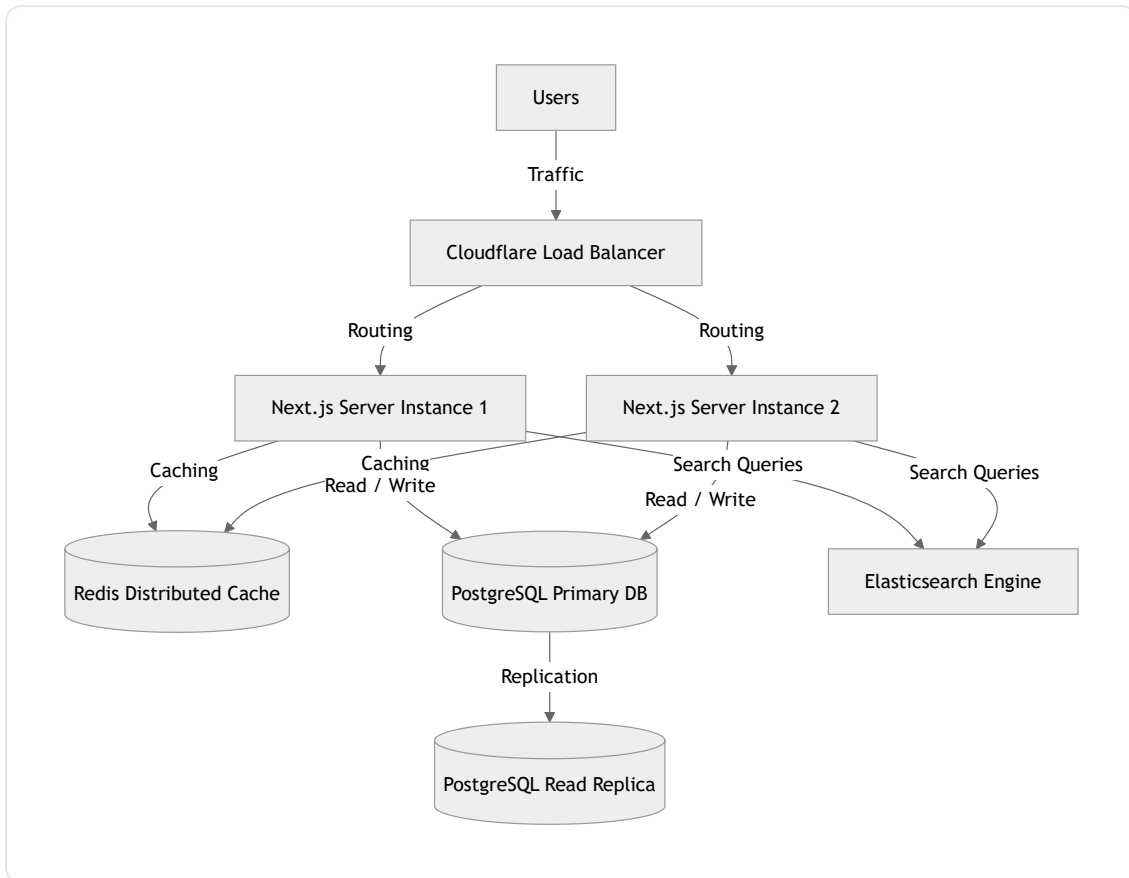
Story 3: Fixing a Redirect Defect in Browse-by-Body-Type

- **Situation:** Clicking on body type filter options on the homepage (like "SUV") redirected users to show vehicles made by a manufacturer named "SUV", returning empty results.
- **Task:** Route the search page to use the correct body type filter parameters.
- **Action:** In `app/page.jsx`, I located the routing link within the Browse-by-Body-Type section and corrected the query parameter from `make` to `bodyType`:

```
-href={` /cars?make=${body.name}`}  
+href={` /cars?bodyType=${body.name}`}
```

- **Result:** Resolved the browse-by-body-type search routing bug.

14. System Design Perspective



Scaling Strategy (1 Million Active Users)

1. **Stateless Compute:** Deploy the Next.js application inside Docker containers across a container cluster (like AWS ECS or Kubernetes).
2. **Load Balancing:** Use a global load balancer (like Cloudflare or AWS ALB) to distribute traffic and handle SSL termination.
3. **Read Replicas:** Scale PostgreSQL database access by routing database reads to read replicas and reserving write actions for the primary database node.
4. **Distributed Caching:** Cache database configurations and vehicle details in a Redis cluster to reduce database query loads.
5. **Search Optimization:** Replace direct SQL database searches with a search index service (like Elasticsearch or Typesense) to handle catalog searches.

15. Project Summary

30-Second Elevator Pitch

"I built **Vehiql Motors**, a full-stack vehicle sales and booking web application using Next.js 15, Prisma, and PostgreSQL. It features an AI-powered image search that uses Gemini 1.5 Flash to identify car makes, colors, and body styles from uploaded photos to filter inventory. The app also includes a dynamic booking system that checks dealership working hours to prevent double-bookings."

2-Minute Explanation

"Vehiql Motors is a full-stack vehicle portal built using Next.js 15, React 19, and Tailwind CSS. The app uses PostgreSQL and Prisma ORM to manage inventory data and bookings. For users, the platform offers an interactive search interface that uses Gemini 1.5 Flash to extract search criteria from uploaded car photos. Users can also select available dates and book test drive slots based on the dealership's working hours. For admins, the application provides an inventory management dashboard that uses optimized, parallelized database queries to calculate sales metrics and conversion rates. Security is handled via Clerk for user authentication and Arcjet middleware for bot detection, rate limiting, and web application firewall protection."

16. Simulated Follow-up Interview Questions

"How would you handle race conditions if two users tried to book the same test drive slot simultaneously?"

- **Answer:** Currently, the system runs a read check before writing the booking:

```
const existing = await db.testDriveBooking.findFirst({ ... });
if (existing) throw new Error("Slot taken");
await db.testDriveBooking.create({ ... });
```

If two users request the same slot at the exact same millisecond, both reads might find the slot empty and proceed to create duplicate bookings. To prevent this, I would add a unique constraint on `[carId, bookingDate, startTime]` in the

database schema. This configuration ensures the database rejects the second write operation, keeping bookings unique.

Note: For the complete 150-question database, please refer to the markdown file saved in your app workspace.